

Twin Edge Coloring in Engineering Applications

J. Anitha¹, R. Manimegalai², Varsha. K³, S. Lokesh⁴, V. Govindan⁵

^{2,3,4} PSG Institute of Technology and Applied Research, Coimbatore-641062, India

² Email:drmm@psgitech.ac.in

¹ Department of Mathematics, Rajalakshmi Institute of Technology ,
Chennai, 600 124, India.

Email:anithaharish78@gmail.com

Abstract

The concept of twin coloring, a variation of graph coloring, focuses on assigning colors to the vertices of a graph such that no two adjacent vertices share the same color and vertices in each color class exhibit structural similarity under graph automorphism. Let $k \geq 2$ be an integer and G be a connected graph of order at least 3. A twin k -edge coloring of G is a proper edge coloring of G that uses colors from Z_k and that induces a proper vertex coloring on G where the color of a vertex v is the sum (in Z_k) of the colors of the edges incident with v . The smallest integer k for which G has a twin k -edge coloring is the twin chromatic index of G and is denoted by $\chi'_t(G)$. This work presents an exhaustive discussion of twin coloring with stress on its mathematics and applications. In this paper, the behavior of twin coloring is studied in different graph topologies. Algorithms are introduced in the literature for calculating twin coloring in different types of graphs, to study its properties, and its consequences for specific graph coloring problems. In addition, visualization methods are proposed to visualize twin-colored graphs, giving easy-to-understand insights into their structural and symmetry behaviors. Experimental outcomes comprise the graphical representation of twin-colored graphs, the twin color class number for specific graph structures, and the algorithm's execution time for different types of graphs. The developed algorithm can handle graphs with any number of vertices, and the results are provided for different graph categories at different scales. This paper also presents the engineering applications of twin coloring such as network analysis, graph compression, understanding redundancy in protein-protein interaction networks, and error detection in networks. Twin coloring provides new techniques for maximizing resource allocation, network construction, and system reliability using graph symmetry and automorphism properties.

Keywords: Twin Coloring, Graph Coloring, Graph Automorphism, Color Class, Structural Similarity, Network Analysis, Graph Symmetry

1 Introduction

Graph theory, as a mathematical discipline, studies graphs as mathematical objects, and has many important applications such as computer science, biology, and social network analysis. Of its many concepts, graph coloring has become a key subject because of its theoretical depth and real-world applications. Graph coloring is the process of coloring the vertices or edges of a graph under some constraints, with vertex coloring being the most studied form. The newest entrant in the list of graph coloring problems is twin coloring, a new idea that has sought to tackle the problems brought by structural symmetry in graphs. Twin vertices with the same adjacency relationship tend to make it hard for conventional coloring methods to work since they bring intrinsic redundancies [1].

1.1 Literature Review

Johnston et al. have introduced the concept of twin edge colorings in graphs, establishing fundamental properties and providing initial classifications for different types of graphs. The work in [4] laid a foundation for further studies in this field by defining the twin chromatic index and presenting conjectures regarding its behavior in various graph structures. Cichacz and Przybyło have expanded on twin edge coloring by incorporating group theory, defining the group twin chromatic index. They investigated how Abelian group structures in the study have [5] influenced edge coloring schemes, offering new bounds

and conjectures applicable to trees and bipartite graphs. Abedin et al. have proposed the concept of improper twin edge coloring, wherein adjacent edges could share colors under certain constraints while maintaining a valid vertex coloring. The study in [6] emphasized the computational complexity and NP-hardness of determining improper twin edge colorings for general graphs. Anantharaman has extended twin edge coloring to total graphs and those with twin chromatic index $\Delta + 2$. The work in [7] novel bounds for total graph twin chromatic indices and introduced an approach for handling graphs with specific structural properties. Rajarajachozhan et al. have examined twin edge colorings in square graphs and product graphs. The work in [8] investigated structural variations in twin chromatic indices across different graph products, revealing new insights into their combinatorial characteristics. Krotov has studied twin edge colorings in infinite square grids, focusing on periodic structures and their impact on coverings. The work in [9] demonstrated how twin colors could be used to model large-scale lattice networks and symmetric graph formations.

Gravier et al. have explored twin vertices in graph theory, analyzing their significance in vertex identification and coloring schemes. The study in [10] provides understanding of combinatorial properties associated with twin structures in various types of graphs. Andrews et al. have revisited the original twin edge coloring concept, further developing results on the chromatic index and its applicability in complex graphs. The study in [11] reinforced earlier theoretical foundations and presented refinements to existing conjectures. Yang et al. have focused on twin edge colorings in infinite paths, examining the implications of extending twin edge colorings to linear infinite structures. The work in [12] focuses on periodicity and structure of infinite graph systems. Bu has examined adjacent vertex-distinguishing edge coloring in planar graphs with girth constraints [13]. The findings provided new conditions under which adjacent vertices remain distinguishable under edge coloring, offering insights into constrained graph coloring problems. Chengchao has explored chromatic numbers for planar graphs with specific girth and degree conditions [14]. The study provided bounds for the adjacent vertex-distinguishing chromatic number and extended understanding of chromatic constraints in planar graphs. Yang et al. have investigated twin edge coloring in infinite lattices, analyzing the impact of twin colors on structured periodic graphs. The work in [12] extended previous findings on infinite graphs and provided new applications for lattice-based network models. Bu has revisited adjacent vertex-distinguishing edge coloring in planar graphs with high girth, reinforcing earlier results and extending them with new conditions for edge coloring constraints [13]. Chengchao has refined his previous work on adjacent vertex-distinguishing chromatic numbers by exploring additional constraints on planar graphs. The study in [14] further solidified the theoretical understanding of chromatic distinctions in complex graph structures.

Yang et al. have contributed additional findings on twin edge coloring in infinite lattices, expanding their previous results and analyzing further combinatorial implication [12]. Bu has continued his work on adjacent vertex-distinguishing edge coloring, focusing on more restrictive planar graph classes and providing further refinements in the field [13]. Chengchao has extended previous results on chromatic numbers of planar graphs, introducing new variations and proving additional constraints for specific graph classes [14]. Yang et al. have further examined twin edge colorings in infinite lattice structures, proposing refined models and proving additional theorems on periodic color assignments [12]. Bu concluded his series of works on adjacent vertex-distinguishing edge coloring by presenting final refinements and applications for structured planar graphs with girth constraints [13]. Engineering applications solved using algorithms such as Power Domination, Chromatic Number are discussed in [15, 16].

Graph Coloring: A proper coloring Let $G = (V, E)$ be a simple graph. A proper vertex coloring of G is a mapping from V to a given set of colors such that adjacent vertices receive different colors. Aliter, a proper edge coloring of G is a mapping from E to a given set of colors such that adjacent edges receive different colors. The minimum number of colors needed for proper vertex coloring indicate proper edge coloring of G are the chromatic number and chromatic index of G and are denoted by $\chi(G)$ and $\chi'(G)$, respectively. Thus, $\chi(G) \leq \Delta(G) + 1$ and $\chi'(G) \geq \Delta(G)$, where $\Delta(G)$ is the maximum degree of G .

Twin Coloring: A twin coloring of a graph is a vertex coloring where every pair of **twin vertices** i.e., vertices having identical neighbor sets except possibly each other, receive the same color.

True-twins: A pair of vertices u and v in a graph are **true-twins** if they are adjacent to each other and share the same set of neighbors. Formally, for vertices u and v , u and v are true-twins if:

$$N(u) = N(v),$$

where $N(x)$ represents the set of neighbors of vertex x .

False-twins: A pair of vertices u and v are **false-twins** if they are **not adjacent** to each other but

have the same set of neighbors. For vertices u and v , u and v are false-twins if:

$$N(u) \neq N(v), \text{ but } |N(u)| = |N(v)|,$$

where $|N(x)|$ denotes the number of neighbors of vertex x .

Twin Graph: A **twin graph** is a graph in which every vertex has at least one twin.

Known Results: Andrews et.al [2, 3] obtained the following results for twin chromatic indices of paths, cycles, complete graphs, and complete bipartite graphs.

1. If n, a, b are integers with $n \geq 3$, $1 \leq a \leq b$ and $b \geq 2$, then

(a). $\chi'_t(P_n) = 3$.

$$(b). \chi'_t(C_n) = \begin{cases} 3 & \text{if } n \equiv 0 \pmod{3} \\ 4 & \text{if } n \not\equiv 0 \pmod{3} \\ 5 & \text{if } n = 5 \end{cases}$$

$$c. \chi'_t(K_n) = \begin{cases} n & \text{if } n \text{ is odd} \\ n+1 & \text{if } n \text{ is even} \end{cases}$$

$$d. \chi'_t(K_{a,b}) = \begin{cases} b & \text{if } b \geq a+2, a \geq 2 \\ b+1 & \text{if } a=1, b \not\equiv 1 \pmod{4} \text{ (or } b=a+1 \geq 3) \\ b+2 & \text{if } a=1, b \equiv 1 \pmod{4} \text{ (or } b=a \geq 2) \end{cases}$$

2. Let T be a tree of order at least 3. Then the following statements hold:

If T is a broom that is not a star, then T has a twin $(\Delta(T) + 1)$ -edge coloring and so $\chi'_t(T) \leq \Delta(T) + 1$.

3. If T is a regular double star, then

$$\chi'_t(T) = \begin{cases} \Delta(T) + 1 & \text{if } \Delta(T) \not\equiv 1 \pmod{4} \\ \Delta(T) + 2 & \text{if } \Delta(T) \equiv 1 \pmod{4} \end{cases}$$

4. If T is an irregular double star, then T has a twin $(\Delta(T) + 1)$ -edge coloring. Then $\chi'_t(T) = \Delta(T) + 1$

5. For each integer $r \geq 3$, the star $K_{1,r}$ has a twin $(r + 2)$ -edge coloring.

6. If T is a regular tree of order at least 6 such that $\Delta(T) \equiv 1 \pmod{4}$, then $\chi'_t(T) = \Delta(T) + 2$.

7. If a connected graph G contains two adjacent vertices of degree $\Delta(G)$, then $\chi'_t(G) = \Delta(G) + 1$. In particular, if G is a connected r -regular graph for some integer $r \geq 2$, then $\chi'_t(G) = \Delta(G) + r$.

8. If G is a connected graph that contains two vertices u and v such that

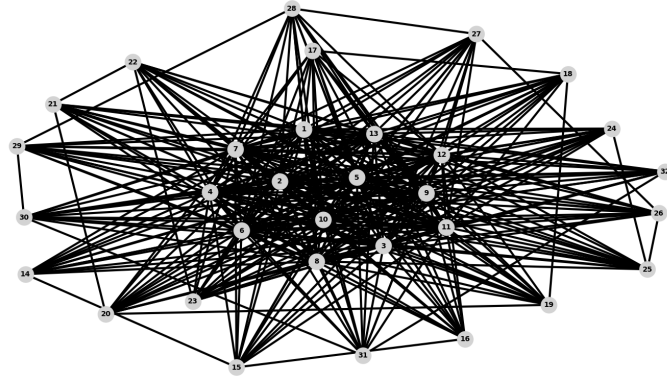
(i). $\deg(u) = \deg(v) = \Delta(G)$;

(ii). both u and v are adjacent with exactly $\Delta(G) - 1$ leaves in G ; and

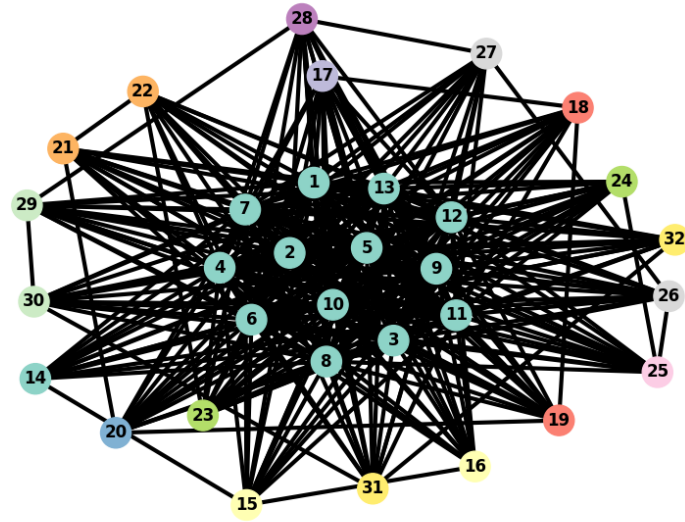
(iii). u and v are adjacent with a common vertex w in G , then $\chi'_t(G) = \Delta(G) + 1$.

1.2 TWIN COLORING IN ENGINEERING APPLICATIONS

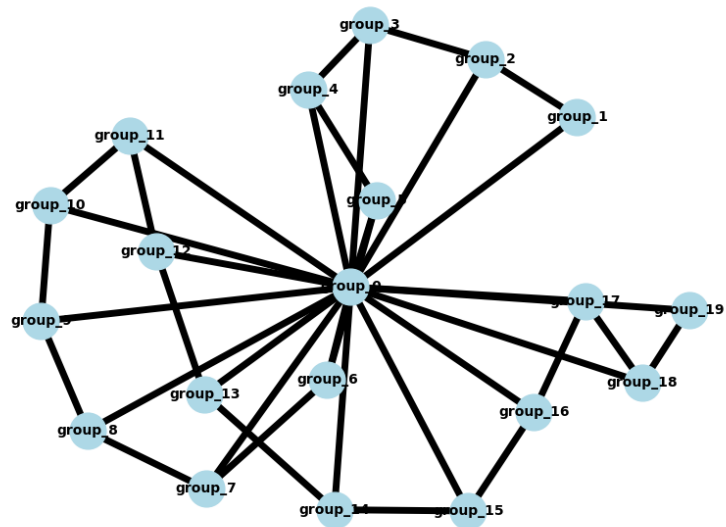
Network Analysis: Twin coloring is a graph-theoretic technique used in network analysis to efficiently classify nodes based on their structural equivalence. In a given network, two nodes are considered as twins if they share identical neighborhoods, either as true-twins having the same neighbors, including each other or false-twins having the same neighbors but not being directly connected. As given in Figure 4(b), twin coloring assigns a unique color to such structurally equivalent nodes, reducing redundancy and enabling efficient compression of network data as shown in Figure 4(c). This method is particularly useful in applications such as community detection, network visualization, and graph isomorphism testing, where reducing the complexity of the graph improves computational efficiency. Researchers can simplify large-scale networks while preserving critical topological properties, by leveraging twin coloring, and thereby making it a valuable tool for optimizing network analysis tasks.



(a) Original Network Connection



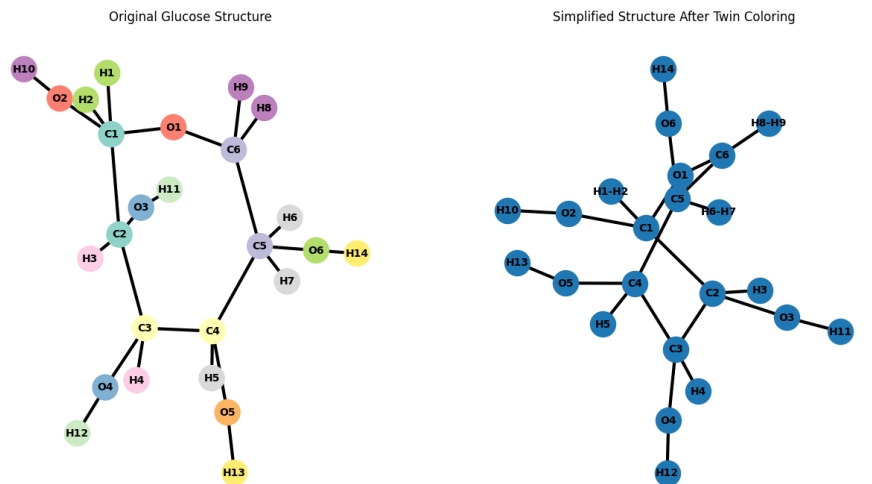
(b) Assigning Colors to Each Network



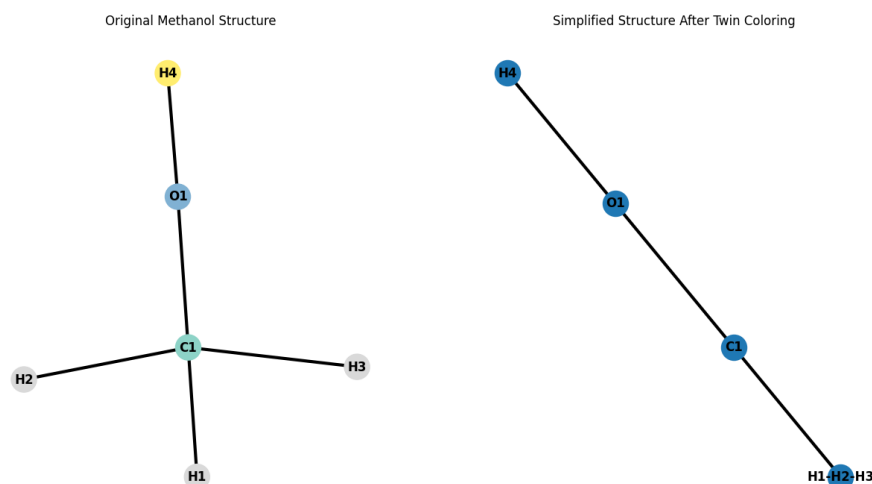
(c) Network After Applying Twin Coloring

Figure 1: Network Analysis using Twin Coloring

Protein-Protein Interaction Analysis: Twin coloring is employed in Protein-Protein Interaction (PPI) analysis to detect structurally similar proteins in interaction networks. In a PPI network, proteins are depicted as nodes and interactions between them as edges as shown in Figure 2(a). Twin coloring labels proteins with the same colors if they have the same interaction patterns and classifies them as true-twins i.e., interacting with each other and having the same neighbors or false-twins i.e., having the same interaction partners but not interacting with each other, as shown in Figure 2(b). This method is useful in detecting redundant proteins, discovering conserved interaction motifs, and reducing network complexity for effective computational analysis. Twin coloring reduces redundancy, improves clustering algorithms, enables functional annotation, and identifies key regulatory proteins more accurately, enhancing the understanding of intricate biological systems.



(a) Protein-Protein Interaction in Glucose

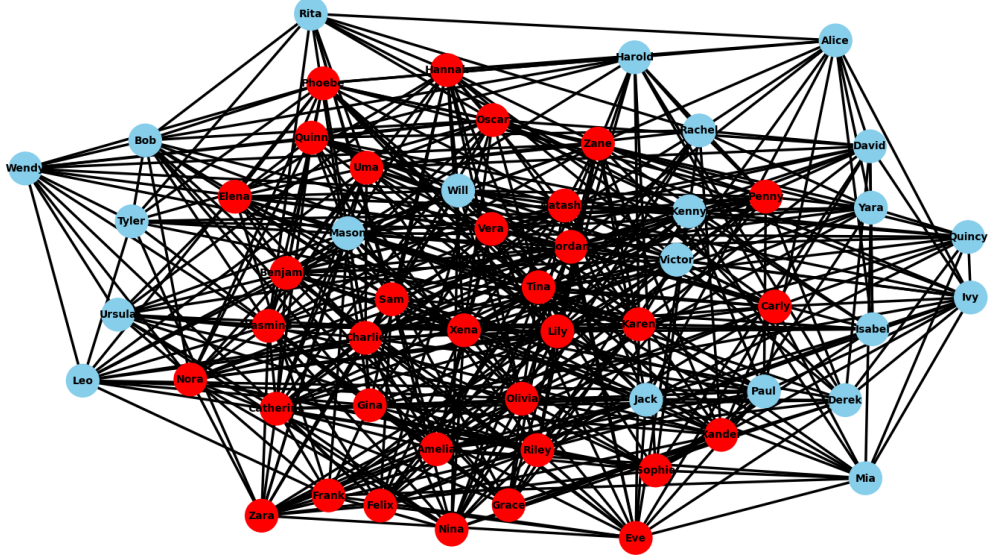


(b) Protein-Protein Interaction in Methanol

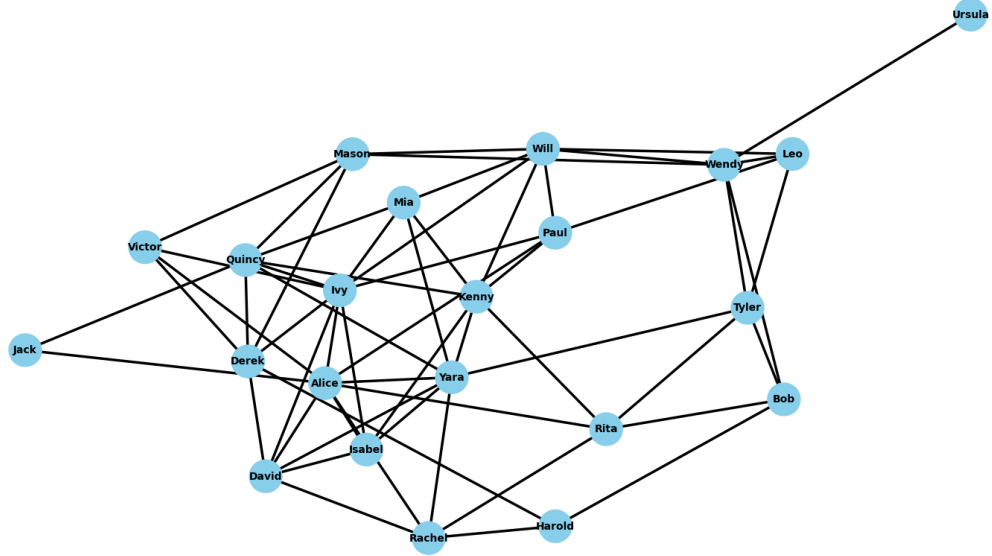
Figure 2: Analysing Protein Interactions using Twin Coloring

Spam Account Detection: Twin coloring can be effectively employed for identifying spam accounts by noting structurally similar or repetitive user profiles in social networks. In a social network graph,

users are vertices or nodes, and their interactions, such as friendships or follows or messages, become edges as shown in Figure 3(a). Relying on the same or extremely similar interaction patterns, spam accounts tend to follow the same legitimate users or to engage with similar content generated by bots. Twin coloring assigns similar, or the same colors to such structurally equivalent accounts, distinguishing true-twins i.e., mutually directly connected spam accounts and false-twins i.e., connected to each other by the same neighbor vertices but without a direct connection as shown in Figure 3(b). With the assistance of twin coloring, social networks can cluster spam accounts and filter them efficiently, saving computation time on their anomaly detection, and provide a higher notch of security against coordinated inauthentic behavior, improving the integrity of online communities.



(a) Typical Social Network Connection with Spam Accounts

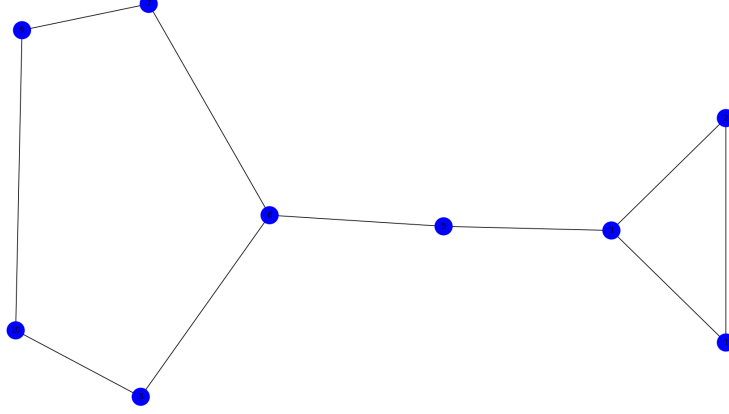


(b) Social Network after Removing Spam Accounts

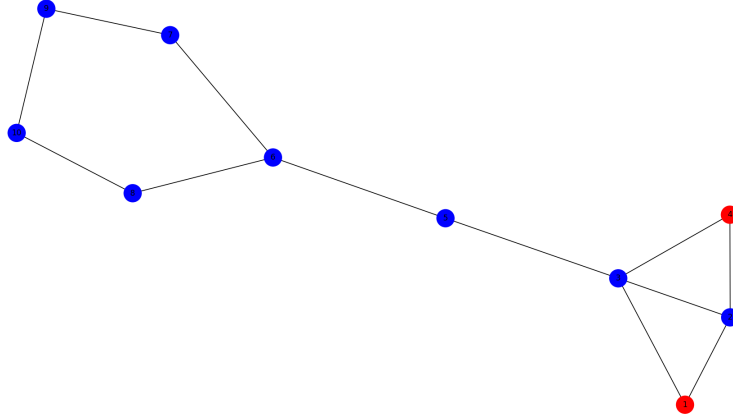
Figure 3: Detection of Spam Accounts in a Social Network

Parallel Processing: Twin coloring is employed in parallel execution of workloads by allowing structurally equivalent computational jobs to be identified for load balancing. In the computational graph, where nodes are computational jobs as shown in Figure 4(a), such a mechanism takes care of assigning

the same color to nodes that receive equivalent connectivity patterns. Specifically, these equivalent nodes of the graph are classified as true and false-twins as shown in Figure 4(b), depending on whether they are mutually reachable with exactly the same neighbors or share the same neighbors and are not connected directly. In general, this twin coloring lends itself to efficient task partitioning into equivalently computed tasks and thereby reduces redundant efforts in computations while maximizing load balancing among parallel processors. Parallel algorithms can effectively reduce inter-process communication overhead, enhance cache locality, and optimize execution efficiency using twin coloring, which is clearly a boon to high-performance computing as well as large-scale data processing applications.



(a) Identifying Twins to Reduce Load Balancing



(b) Eliminating Twins to Optimize Load Balancing

Figure 4: Performing Twin Coloring to Optimize Parallel Processing

2 IMPLEMENTATION OF TWIN COLORING ALGORITHM

In this paper, an effective algorithm is developed to determine the twin coloring of a graph by detecting and clustering twin vertices i.e., nodes having the same neighborhoods. The algorithm is organized into several modules, such as a Graph Representation module, a Twin Detection module, and a Color Assignment module, to provide flexibility and ease of implementation. Figure 5 depicts the general algorithm for twin coloring. The algorithm starts with initializing an empty color mapping for all vertices and calculating the twin set, which is all vertex pairs with the same neighbors. The vertices are sorted in non-increasing order of degree to optimize the coloring order. The algorithm processes each vertex, assigning it the smallest available color and propagating the same color to all its twin vertices, so that similar nodes get the same label. A color counter is used to count unique color assignments, and the process continues until all vertices are colored. After completion, the algorithm returns the twin coloring assignment, which minimizes the number of different colors used while maintaining twin relationships. The execution time of the algorithm is calculated by finding the difference between the start and end

times, ensuring efficiency in large graphs. Visualization techniques are incorporated to visualize the coloring process, providing insight into the topology-graph relationship and twin chromatic properties. The proposed method guarantees accurate and computationally efficient twin coloring, making it highly useful in network clustering, graph isomorphism detection, and pattern recognition applications.

Algorithm Twin Coloring

```

1: Initialize an empty color map  $C : V \rightarrow \emptyset$ 
2: Compute the Twin Set  $T = \{(u, v) \in V \times V \mid N(u) = N(v)\}$ 
3: Sort  $V$  in non-increasing order of degree
4: Initialize color counter  $c \leftarrow 1$ 
5: for each vertex  $v$  in  $V$  do
6:   if  $v$  is not colored then
7:     Assign  $C(v) \leftarrow c$ 
8:     Find all twin vertices of  $v : T_v = \{u \in V \mid (u, v) \in T\}$ 
9:     for each  $u$  in  $T_v$  do
10:      Assign  $C(u) \leftarrow c$ 
11:     end for
12:     Increment  $c \leftarrow c + 1$ 
13:   end if
14: end for
15: return  $C$ 

```

▷ Twin coloring assignment

Figure 5: Algorithm for Twin Coloring of Graph Vertices

3 Algorithm for Twin-Edge Coloring

In this section, we obtain an algorithm using Python code with graph visualization for the path, cycle, and Star graphs, which is shown by Andrews et al. [2, 3]. If a connected graph G contains two adjacent vertices of degree $\Delta(G)$, then $\chi'_t(G) = \Delta(G) + 1$.

3.1 Algorithm for Path P_n

Input: Path $P_n, n \geq 2$.

Algorithm:

(i). Label the vertices of P_n from left to right as $v_1, v_2, \dots, v_n$ and the edges $e_1, e_2, \dots, e_{n-1}$.

(ii). Assign colors $c : E(P_n) \rightarrow Z_k, k \in \{0, 1, 2\}$.

(iii). Assign $c(e_i) = \begin{cases} 0 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 2 & \text{if } i \equiv 2 \pmod{3} \end{cases}$

(iv). Assign $c'(v_i) = \begin{cases} 2 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 0 & \text{if } i \equiv 2 \pmod{3} \end{cases}$

Output : $\chi'_t(P_n) = 3$.

Proof of Correctness: Colors assigned for edges, $c(e_i), 1 \leq i \leq n$ induces a proper edge coloring and colors assigned for vertices, $c'(v_i), 1 \leq i \leq n$ induces a proper vertex coloring. Hence, colors assigned for the vertices and edges induces a twin edge coloring of P_n . Hence $\chi'_t(P_n) = 3$.

3.2 Algorithm for Cycle C_n

Input: Cycle graph $C_n, n \geq 2$.

Algorithm: (i) Label the vertices of P_n from left to right as $v_1, v_2, \dots, v_n$ and the edges $e_1, e_2, \dots, e_n$.

(ii). Assign colors $c : E(C_n) \rightarrow Z_k, k \in \{0, 1, 2\}$

$$\begin{aligned}
& (iii) \text{ For } n \equiv 0 \pmod{3}. \text{ Select } c(e_i) = \begin{cases} 0 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 2 & \text{if } i \equiv 2 \pmod{3} \end{cases} \\
& (iv) Assign $c'(v_i) = \begin{cases} 2 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 0 & \text{if } i \equiv 2 \pmod{3} \end{cases} \\
& (v) For $n \not\equiv 0 \pmod{3}$. Assign $c(e_i) = \begin{cases} 0 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 2 & \text{if } i \equiv 2 \pmod{3}, 1 \leq i \leq n-2 \end{cases} \\
& (vi). Assign $c(e_{n-1}) = 3, c(e_n) = 0$. \\
& (vii). Assign $c'(v_i) = \begin{cases} 2 & \text{if } i \equiv 0 \pmod{3} \\ 1 & \text{if } i \equiv 1 \pmod{3} \\ 0 & \text{if } i \equiv 2 \pmod{3}, 1 \leq i \leq n-1 \end{cases} \\
& (vi). Assign $c'(v_{n-1}) = 3$. \\
& \textbf{Output: } \chi'_t(C_n) = \begin{cases} 3 & \text{if } n \equiv 0 \pmod{3} \\ 4 & \text{if } n \not\equiv 0 \pmod{3} \\ 5 & \text{if } n = 5 \end{cases}
\end{aligned}$$

Proof of Correctness : Colors assigned for edges, $c(e_i), 1 \leq i \leq n$ induces a proper edge coloring and colors assigned for vertices, $c'(v_i), 1 \leq i \leq n$ induces a proper vertex coloring. Hence, colors assigned for

the vertices and edges induces a twin edge coloring of C_n . Hence, $\chi'_t(C_n) = \begin{cases} 3 & \text{if } n \equiv 0 \pmod{3} \\ 4 & \text{if } n \not\equiv 0 \pmod{3} \\ 5 & \text{if } n = 5 \end{cases}$

3.3 Algorithm for Star Graph $S_{1,k}$

Input: Star Graph $S_{1,k}, k \geq 2$.

Algorithm:

(i) Label the vertices of $S_{1,k}$, the center node as v_n leaf node as $v_1, v_2, \dots, v_k$ and pendant the edges $e_1, e_2, \dots, e_k$.

(ii) Assign $c(e_i) : E(S_{1,k}) \rightarrow i, i = 0, 1, \dots, k-1$ to the pendent edges.

(iii) Assign $c'(v_i) : V(S_{1,k}) \rightarrow i, i = 0, 1, \dots, k-1$

(iv) $c'(v_0) = \sum_{i=0}^k i$ to the center node.

Output: $\chi'_t(S_{1,k}) = k$.

Proof of Correctness: Colors assigned for edges, $c(e_i), 0 \leq i \leq k-1$ induces a proper edge coloring and colors assigned for vertices, $c'(v_i), 0 \leq i \leq k-1$ induces a proper vertex coloring. Hence, colors assigned for the vertices and edges induces a twin edge coloring of $S_{1,k}$. Hence $\chi'_t(S_{1,k}) = k$.

3.4 Algorithm for Binary Tree T_l

Input: Binary Tree $T_l, l \geq 2$.

Algorithm:

(i) Label the vertices of T_l , level wise from left to right starting from the root vertex as $0, 1, 2, \dots, 2^l - 1$.

(ii) Assign colors that start from level 1, left to right $c(e_i) : E(T_i) \rightarrow \{1, 2\}, i = 1, 3, \dots, l-1$.

(iii) Assign colors that start from level 2, left to right $c(e_i) : E(T_i) \rightarrow \{0, 3\}, i = 2, 4, \dots, l$.

(iv) Assign $c'(v_i) : V(T_i) \rightarrow \{0, 1\}, i = 1, 3, \dots, l-1$.

(v) Assign $c'(v_i) : V(T_i) \rightarrow \{3, 2\}, i = 2, 4, \dots, l$.

Output: $\chi'_t(S_{1,k}) = 4$.

Proof of Correctness: Colors assigned for edges, $c(e_i), 0 \leq i \leq l$ induces a proper edge coloring and colors assigned for vertices, $c'(v_i), 0 \leq i \leq l$ induces a proper vertex coloring. Hence, colors assigned for the vertices and edges induces a twin edge coloring of $S_{1,k}$. Hence $\chi'_t(S_{1,k}) = 4$.

Graph	Input	Output
Star Graph $S_{1,k}$	Star graph $S_{1,k}$ with $k + 1$ nodes	$\chi'_t(S_{1,k}) = k$.
Cycle graph C_n	Cycle graph of length n	$\chi'_t(C_n) = \begin{cases} 3 & \text{if } n \equiv 0 \pmod{3} \\ 4 & \text{if } n \not\equiv 0 \pmod{3} \\ 5 & \text{if } n = 5 \end{cases}$
Path P_n	Path graph of length n	$\chi'_t(P_n) = 3$
Binary Tree T_l	Binary tree with nodes	$\chi'_t(T_l) = 4$

Table 1: Algorithms for Coloring Various Types of Graphs

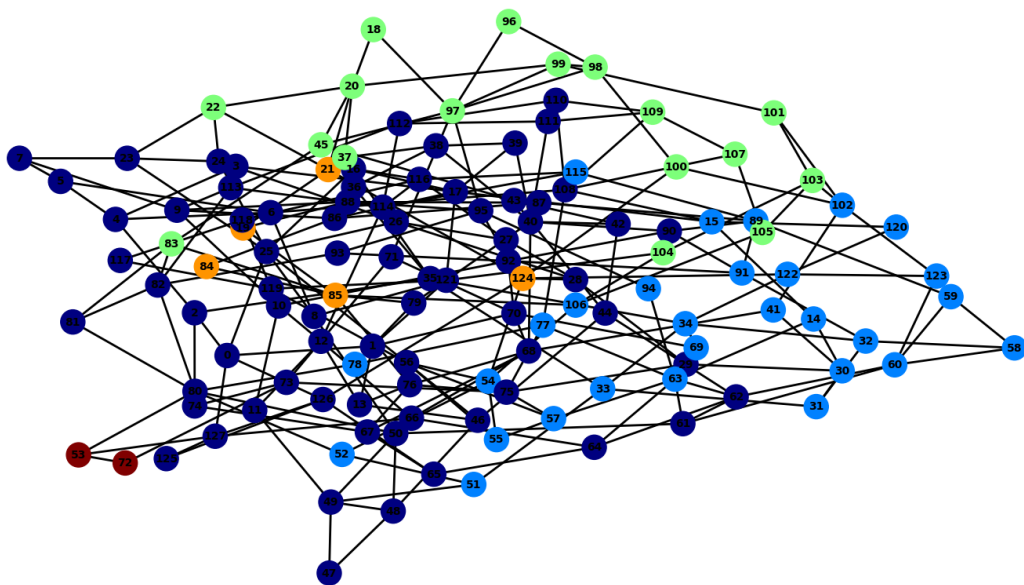
4 EXPERIMENTAL RESULTS

The proposed twin coloring algorithm handles different types of graphs, namely connected graphs, star graphs, fan graphs, cycle graphs, path graphs, binary trees, and sibling trees. It calculates the chromatic number and execution time for different types of graphs with efficiency for various numbers of nodes i.e., 128, 256, 512, and 1024 nodes. As indicated in Table 2, the chromatic number is the same for all types of graphs except connected graphs, where its value changes slightly with structure, e.g., 4, 4, 5, and 4 for tested sizes. Execution time grows with the number of nodes, with bigger times for more complex types of graphs, especially the connected graph, since its structure affects computational complexity. The algorithm also provides visualization, which supports better understanding of the chromatic number by presenting different node colors with each color class. This function is especially useful for educational and practical applications, such as exploring scheduling issues or network routing. It should be noted that (i) Star Graphs and Path Graphs always need only 2 colors. (ii) Fan Graphs have a chromatic number of 3 because of their cycle and path structure. (iii) Cycle Graphs, Binary Trees, and Sibling Trees continue to have a chromatic number of 2, which is because they are bipartite in structure. (iv) Connected Graphs have varying chromatic numbers based on their structure and edge density.

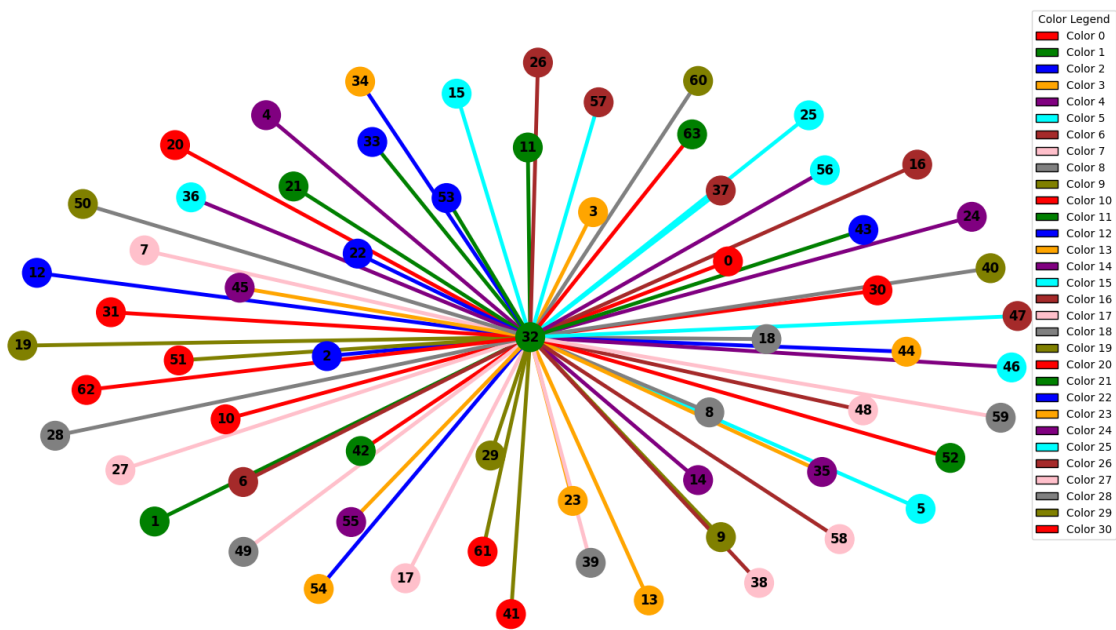
The execution time for the algorithm grows with the number of nodes proportionally, with a significant jump for graphs such as connected graphs and fan graphs. Notably, the average execution time for different number of nodes in a connected graph ranges from 1.98 sec to 10.85 sec. The average execution time for different number of nodes in a star graph ranges from 2.42 sec to 12.67 sec. The average execution time for different number of nodes in a fan graph ranges from 2.04 sec to 12.84 sec. The average execution time for different number of nodes in a cycle graph ranges from 2.29 sec to 4.12 sec. The average execution time for different number of nodes in a path graph ranges from 2.08 sec to 12.65 sec. The average execution time for different number of nodes in a binary tree ranges from 2.04 sec to 13.17 sec. The average execution time for different number of nodes in a sibling tree ranges from 2.21 sec to 12.75 sec. This is because more computational effort is required for these structures. The use of GPU-based processing could further minimize execution time, especially for denser and larger graphs. Applications of the chromatic number have a broad spectrum across areas such as scheduling, network analysis, and parallel processing, because efficient algorithms for graph coloring are crucial. The experimental results confirm the robustness and scalability of the algorithm with different graph structures.

Types of graphs	Execution time for nodes (sec)				Chromatic Number
	128	256	512	1024	
Connected Graph	1.98	2.76	6.48	10.85	4, 4, 5, 4
Star Graph	2.42	2.52	6.18	12.67	2
Fan Graph	2.04	2.29	6.37	12.84	3
Cycle Graph	2.29	2.44	2.56	4.12	2
Path Graph	2.08	2.32	6.11	12.65	2
Binary Tree	2.04	2.65	6.28	13.17	2

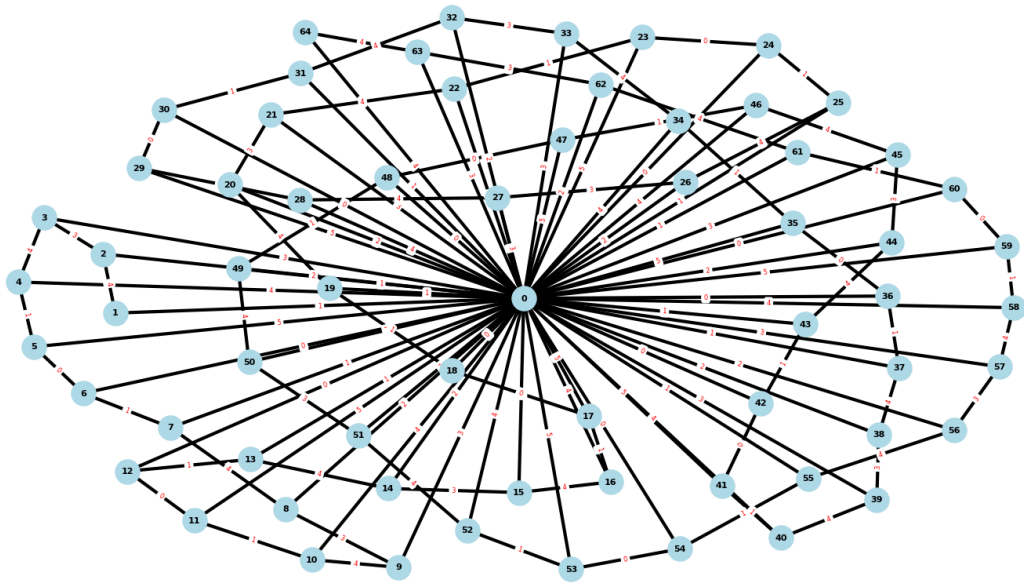
Table 2: Execution Time and Chromatic Number for Various Network of Nodes



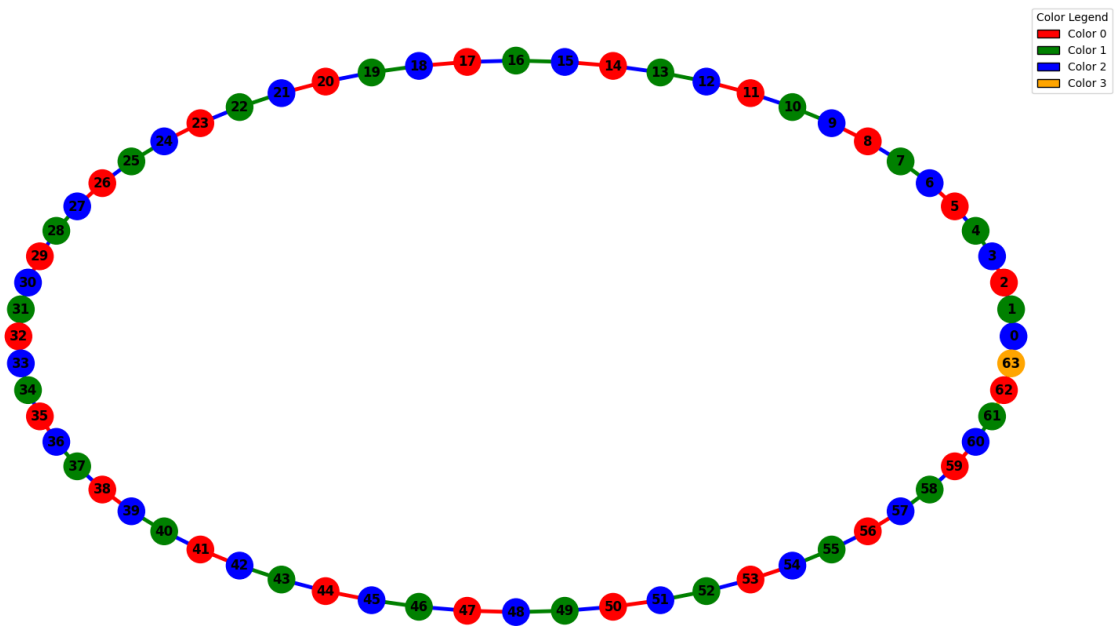
(a) Connected Graph



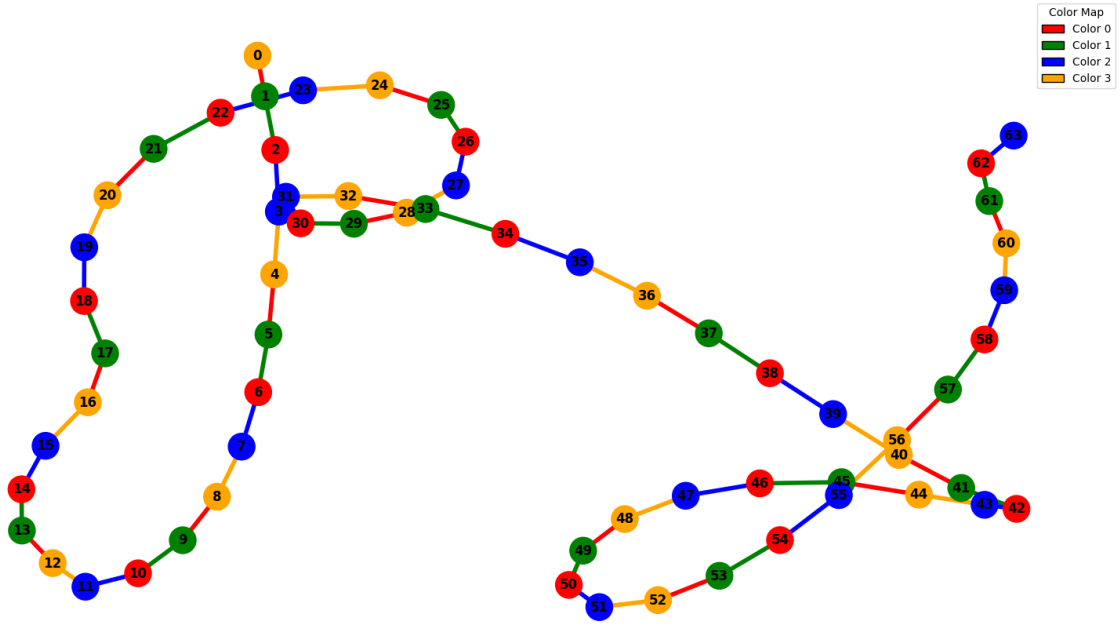
(b) Star Graph



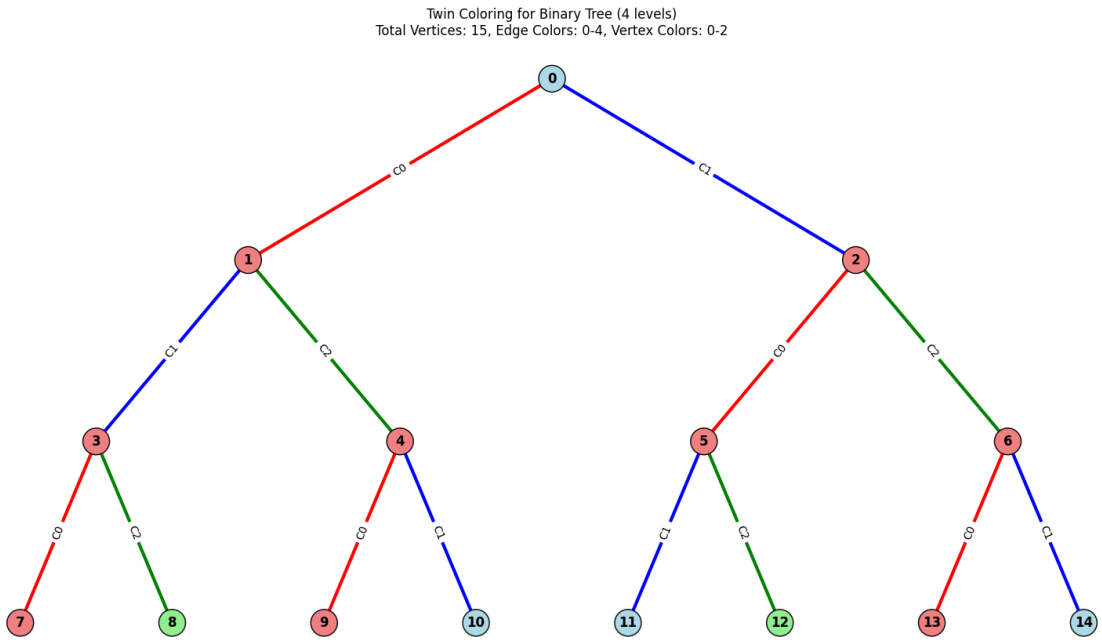
(c) Fan Graph



(d) Cycle Graph



(e) Path Graph



(f) Binary Tree

Figure 6: Execution for a Twin Chromatic Number Index

4.1 EXPERIMENTAL RESULTS OF ENGINEERING APPLICATIONS

Table 3 computes the execution time for the engineering applications discussed in section 1.1.

Application	Description	Execution Time with and without Twin Nodes (sec)	Improvement
Network Analysis	Twin coloring partitions structurally equivalent nodes by assigning distinct colors to true and false-twins, thus mitigating redundancy for possible compression of networks. This achieves higher computational efficiencies in cases such as community detection, graph isomorphism, and visualization while at the same time conserving major topological properties.	0.0072, 0.0035	50%
Protein-Protein Interaction Analysis	Twin coloring analysis of Protein-Protein Interaction (PPI) identifies structurally conserved proteins by assigning the same color to proteins that have similar interaction patterns. This technique categorizes proteins in terms of true-twins and false-twins. Therefore, it will help in redundancy detection, identification of conserved motifs, and simplification of the interaction network, which will in turn improve clustering, functional annotation, and regulatory protein discovery for better biological system analysis.	0.004, 0.002	50%
Spam Account Detection	Twin coloring detects social network spam through the identification of structurally equivalent user profiles, which draw a clear line between true and false-twins. It clusters and filters spam efficiently through an intensified detection of coordinated inauthentic behavior, enhancing anomaly detection and bolstering online security.	0.0026, 0.000098	4%
Parallel Processing	Twin coloring helps in enhancing parallel execution with the aid of determining structurally equivalent tasks in order to equally distribute workloads. It minimizes redundant computation, optimizes task partitioning, works towards better cache locality, thus being invaluable in high-performance computation and large-scale data processing.	0.025, 0.004	16%

Table 3: Comparison based on Implementation Time

5 CONCLUSIONS AND FUTURE WORK

This work formulates the use of algorithms for computing twin coloring over various classes of graphs, such as connected graphs, cycle graphs, path graphs, fan graphs, star graphs, sibling trees, and binary trees. For each class of graphs, a step-by-step description of the twin coloring algorithm followed by a clear description of color assignment is given. The twin coloring execution time is measured for various graph structures, and the chromatic number is calculated and tabulated for graphs with varying numbers of nodes. Apart from computational findings, a visualization is produced for each class of graphs, where various colors for nodes represent the corresponding color classes, thus facilitating better understanding of the algorithmic solution. The computations show that the twin coloring algorithm

is scalable and robust, as it solves graphs with nodes up to 5000 in number with ease. The twin coloring idea presented here has promising applications for solving real-world problems, i.e., scheduling, resource allocation, and network optimization. Map coloring, compiler register allocation, and timetable scheduling in schools are important applications. Future research can extend the methodology to include more complex graph structures, for instance, hypergraphs or irregular graphs, and explore its implications in new application areas like AI-based optimization and IoT-based network design. The synergy between efficient computation and visualization provided by this twin coloring method underscores the importance of addressing theoretical and real-world problems of graph theory and its numerous applications.

References

- [1] Graph Theory Modeling, Applications and Algorithms, Geirr Agnarsson and Raymond Greenlaw
- [2] E. Andrews, L. Helenius, D. Johnston, J. VerWys, and P. Zhang, On twin edge colorings of graphs, *Discuss. Math. Graph Theory*, 34 (2014), 613 – 627.
- [3] E. Andrews, D. Johnston, and P. Zhang, On twin edge colorings in trees, *J. Combin. Math. Combin. Comput.*, 94 (2015), 115 – 131.
- [4] Johnston, D., VerWys, J., & Zhang, P., "Twin Edge Colorings of Graphs" *Journal of Discussiones Mathematicae Graph Theory*, vol. 3, pp. 615-623, 2014.
- [5] Cichacz, S., & Przybyło, J., "Group Twin Coloring of Graphs", *Journal of Discrete Mathematics and Theoretical Computer Science*, vol. 20, no. 1, 2017.
- [6] Abedin, P., Akbari, S., Demange, M., & Ekim, T., "Improper Twin Edge Coloring of Graphs", 2016
- [7] Anantharaman, S., "Twin Edge Coloring of Total Graph and Graphs with Twin Chromatic Index $\Delta + 2$ ", *Journal of Applications and Applied Mathematics*, vol. 15, no. 1, pp. 314-324, 2020.
- [8] Rajarajachozhan, S., Anantharaman, S., & Somasundaram, S., "Twin Edge Colorings of Certain Square Graphs and Product Graphs", *Electronic Journal of Graph Theory and Applications*, vol. 4, no. 1, pp. 172-182, 2016.
- [9] Krotov, D. S., "Perfect Colorings of the Infinite Square Grid: Coverings and Twin Colors", vol. 30, no. 2, 2020.
- [10] Gravier, S., Schmidt, S., & Slimani, S., "Twins and Vertex Identification on Graphs", 2017.
- [11] Andrews, E., Helenius, L., Johnston, D., VerWys, J., & Zhang, Ping., "Twin Edge Colorings of Graphs", *Journal of Discussiones Mathematicae Graph Theory*, vol. 3, pp. 613-627, 2014.
- [12] Yang, H., Tian, S., Jiao, S., & Cai, X, "Twin Edge Colorings of Infinite Paths", *Journal of Shandong University (Natural Science)*, vol. 52, no. 11, 2017.
- [13] Bu, Y, "Adjacent Vertex-Distinguishing Edge Coloring of Planar Graphs with Girth at Least 6" *Journal of Lanzhou University (Natural Sciences)*, vol. 31, no. 4, pp. 429-439, 2011.
- [14] Chengchao Yan, Danjun Huang, Dong Chen, Weifan Wang, "Adjacent vertex distinguishing edge colorings of planar graphs with girth at least five", *Journal of Combinatorial Optimization*, vol. 35, no. 2, pp. 454-462.
- [15] J. Anitha, R. Manimegalai, S. Nithishkumar, R. Arthi, V. Govindan, Siriluk Donganont, "Optimal Placement of PMUs using k-Power Domination Number in Grid Networks", *European Journal of Pure and Applied Mathematics*, Vol. 18, Issue 1, 5753, 2025.
- [16] R. Manimegalai, J. Anitha, J. Dharineesh, M. Chandramouleeswaran, "Graph Representation and Chromatic Number Computation for Engineering Applications", *International Conference on Smart System for Integrated Computing and Communication*, Springer LNICST Book Series, 2024.